HƯỚNG DẪN THỰC HÀNH · NHÓM 10

Đọc cây, chạy mã, giải thích quyết định

Nguyễn Bích Vân – IAI–UET–VNU

Học phần: Biểu diễn tri thức và tìm kiếm nâng cao

Nhóm 4 sinh viên · 10 ngày · Minimax và Alpha–Beta

Mã đã hoàn chỉnh. Bắt đầu từ ví dụ 4 lá, sau đó thay list utility để chạy cây của nhóm. Các ví dụ có lời giải ở đây dùng dữ liệu làm quen; không phải bảng đáp án của các bài tính tay.

1. Mở đúng thư mục trong VS Code

Giải nén toàn bộ ZIP. Mở `GiuaKy_Nhom10_CayTroChoi_4SV_10Ngay`; các tệp `chay_mau.py`, `search.py`, `tree_model.py` nằm ngay trong thư mục đó. Không gõ `cdpython`. Mở Terminal tại thư mục này; dùng Python/Anaconda 3.10 trở lên, không cần cài gói bổ sung.

```
python kiem_tra_moi_truong.py
python chay_mau.py
python minimax_alphabeta_toi_gian.py
python -m unittest -v test_mau test_nhom_mau
```

Nếu báo `can'topenfile`, kiểm thư mục Terminal trước khi sửa code. Đường dẫn có khoảng trắng phải đặt trong dấu ngoặc kép. Dấu `>>>` là trình nhập Python; gõ `exit()` rồi mới chạy lệnh Terminal.

2. Ví dụ: list [7,2,5,9] có nghĩa gì?

Gốc MAX, tầng con MIN; nhánh trái có 7 và 2, nhánh phải có 5 và 9. Vì vậy hai con trả 2 và 5, gốc trả 5 và chọn B. Cả hai thuật toán đều phải cho kết quả này.

```
LEAVES = [7, 2, 5, 9]
ROOT_PLAYER = "MAX"
ALGORITHM = "alphabeta"
ORDER = "LR"
```

Đây là bốn biến cần sửa trong `chay_mau.py`; không sửa vòng lặp tìm kiếm chỉ để đổi số ở lá. Thử tự tính trước khi thay 9 bằng −3 rồi chạy lại.

3. Biểu diễn và tạo cây từ dữ liệu

Dữ liệu vào đơn giản

Mỗi cây là cây nhị phân đầy đủ: số lá $1, 2, 4, 8, 16, \ldots, 256$. Một list 16 số tạo bốn tầng cạnh, tổng 31 nút. Hai nửa list tương ứng hai cây con; tiếp tục chia đôi đến từng lá. MAX/MIN đổi luân phiên, không phụ thuộc dấu của điểm.

```
from tree_model import make_tree
from search import solve
leaves = [7, 2, 5, 9]
root = make_tree(leaves, root_player="MAX")
answer = solve(root, algorithm="minimax", order="LR")
print(answer["value"], answer["action"])
```

Đọc lớp Node

```
@dataclass(frozen=True)
class Node:
    name: str
    player: Optional[str]
    children: tuple["Node", ...] = ()
    utility: Optional[int] = None
```

Nút trong có tên, người chơi và hai con. Lá có utility nguyên, player=None, children rỗng. `frozen=True` giúp ngăn sửa trực tiếp đối tượng đã tạo. List đầu vào được chép sang tuple.

Tên nút và thứ tự

Root là R; hai con A/B; sau đó A1/A2/B1/B2; rồi A11/A12,... Lá tên L01,L02,... từ trái sang phải. Nếu cây chỉ có 2 lá, action ở gốc là L01 hoặc L02. RL đảo thứ tự đọc, không đổi nhãn.

Những dữ liệu bị từ chối

Số lá không phải lũy thừa của 2; list rỗng; chuỗi hoặc số thực thay cho utility nguyên; giá trị boolean; người chơi ngoài MAX/MIN. Mỗi utility có trị tuyệt đối không quá 10^6. Dữ liệu nhóm trong đề dùng khoảng nhỏ hơn $[-20, 30]$.

`make_tree` tạo cây đầy đủ từ list, không phải giải bài. Không nhập list giá trị đã tính cho nút trong: thuật toán sẽ tự tính từ lá lên.

4. Minimax: hỏi con rồi mới chọn ở cha

Bản tối giản trong `minimax_alphabeta_toi_gian.py` dùng list lồng nhau để nhìn rõ đệ quy:

```
def minimax(tree, is_max=True):
    if type(tree) is int:
        return tree
    scores = [minimax(child, not is_max) for child in tree]
    return max(scores) if is_max else min(scores)
```

`tree=[[7,2],[5,9]]` không giống list phẳng của `make_tree`; đây là hai cách biểu diễn trong hai ví dụ, không đổi lẫn nhau.

Bản đầy đủ còn cần trả hành động

```
if node.utility is not None:
    return node.utility, None
is_max = node.player == "MAX"
best = -math.inf if is_max else math.inf
action = None
for child in children:
    score, _ = visit(child, alpha, beta, depth + 1)
    better = score > best if is_max else score < best
    if better:
        best, action = score, child.name
return best, action
```

Dấu gạch dưới bỏ hành động sâu hơn của con; cha chọn **con trực tiếp**, không chọn ngay một lá ở xa. Sau khi con trả, chương trình quay lại vòng for đang dở ở cha. Mỗi lần như vậy là một bước quay lui.

Tự kiểm tra trước khi thêm cắt nhánh

Chạy cây 1 lá bằng 0, hai lá đều âm, gốc MIN và hai lá hòa. Gốc là lá có action=None, không phải một chuỗi ký tự “None”. Dùng `isnotNone` khi kiểm lá: utility 0 vẫn là dữ liệu hợp lệ.

Theo dõi trong VS Code

Đặt breakpoint ở dòng gọi `visit(child,...)`. F11 đi vào con; F10 thực hiện một dòng; nhìn các biến node, best, action, alpha, beta và Call Stack. Khi dùng debugger, không cần bật `--step` cùng lúc.

5. Alpha–Beta: thêm cửa sổ, không đổi mục tiêu

Sau khi nhận điểm con và cập nhật best/action, bản đầy đủ thêm đoạn:

```
stop = best >= beta if is_max else best <= alpha
if stop:
    skipped = [c.name for c in children[index + 1:]]
    if skipped:
        stats["cutoffs"] += 1
        stats["pruned_branches"] += len(skipped)
    break
if is_max:
    alpha = max(alpha, best)
else:
    beta = min(beta, best)
```

Đoạn này chỉ bật cho Alpha–Beta. Không reset alpha/beta sau mỗi con. Ở MAX, best cục bộ có thể nhỏ hơn alpha kế thừa; dùng max giữ ngưỡng đã biết, không gán alpha=best vô điều kiện.

Hai dấu so sánh phục vụ hai mục đích

Chọn hành động: tốt hơn hẳn ($>$ hoặc $<$).

Cắt nhánh: bằng ngưỡng cũng cắt ($\geq$ hoặc $\leq$).

Không đồng nhất hai quy tắc. Một số trả về do cắt có thể chỉ là cận, không đủ để kết luận hành động mới hòa thật với hành động cũ.

Đọc kết quả và trace

ENTER: vào nút; LEAF: đọc utility; UPDATE: con vừa trả, cha cập nhật; PRUNE: bỏ con chưa gọi; RETURN: nút trả lên. `--step` in sự kiện và gọi input ngay lúc đệ quy thực thi; nhấn Enter tại Terminal, Ctrl+C để dừng.

```
python run.py --case T0 --algo alphabeta --trace --tag trace_T0
python run.py --case T1 --algo alphabeta --step --tag step_T1
python run.py --case T0 --algo minimax --tag T0_minimax
```

Những số vô hạn được lưu thành chuỗi `+inf/-inf` trong JSON. `unvisited_nodes` được liệt kê sau tìm kiếm bằng cấu trúc đầu vào; thao tác đó không được tính là thuật toán đã thăm hoặc đọc utility của nút.

`returns` của Alpha–Beta có nhãn EXACT, UPPER, LOWER theo cửa sổ lúc vào; không chép mọi số đó vào bảng "giá trị thật". Đường đầy đủ `principal_variation` chỉ được cung cấp trong lần Minimax, không tự dựng từ các cận.

6. Chạy ma trận và thay dữ liệu riêng

24 cấu hình chung và trace riêng

```
python run.py --batch --tag bat_buoc
python run.py --case T0 --t 11 --algo alphabeta --trace --tag t11
```

Batch gồm 12 cơ bản, 10 tham số, 2 affine. Không cần nhập 24 lần. Giá trị –t thay riêng L01 của T0; không biến T1/T2 thành cây tham số. Tên tag khác nhau để không ghi đè kết quả cần giữ.

Tạo hai cây riêng cho nhóm

```
python tao_du_lieu.py --seed 1026
python run.py --data du_lieu_nhom.json --tag tu_tao
```

Nhóm chọn seed riêng. Công cụ tạo N1 (16 lá, gốc MAX), N2 (8 lá, gốc MIN), lưu `du_lieu_nhom.json`. Khi tệp đã tồn tại, đổi tên hoặc dùng –overwrite có chủ ý. Có thể thay số trong JSON sau khi đã dự đoán.

Ví dụ cấu trúc JSON để đọc

```
{
  "trees": [
    {"name": "Demo", "root_player": "MAX", "leaves": [7,2,5,9]}
  ]
}
```

Ví dụ 4 lá này chỉ để học định dạng; dữ liệu nộp phải là N1/N2 đúng kích thước và khoảng điểm của đề. Không có dấu phẩy thừa; số viết không có ngoặc kép.

Đọc CSV và JSON

CSV có mỗi dòng một cấu hình với value/action và các số đếm. JSON còn có danh sách lá đọc, mọi nút chưa thăm và bảng returns. Không lấy `pruned_branches` làm số lá bỏ. Mọi tệp output nằm cạnh runner, không tùy vào nơi Python được cài.

Muốn so thời gian: `pythonrun.py--batch--repeat3--tagtiming`; ba lần là lặp trong cùng cấu hình, không phải 72 cấu hình. Tắt trace/step khi đo, không so các máy khác nhau. Phân tích thời gian không bắt buộc.

7. Viết kiểm thử, điền báo cáo, nộp bài

Một test độc lập từ phép tính tay

Tạo `test_nhom.py`, dùng `test_nhom_mau.py` làm mẫu cấu trúc, thay tình huống và lập kỳ vọng riêng:

```
import unittest
from tree_model import make_tree
from search import solve
class MyTests(unittest.TestCase):
    def test_vi_du(self):
        # min(7,2)=2; min(5,9)=5; goc MAX chon B.
        root = make_tree([7,2,5,9])
        for algo in ("minimax", "alphabeta"):
            r = solve(root, algo)
            self.assertEqual((r["value"], r["action"]), (5,"B"))
if __name__ == "__main__":
    unittest.main()
```

Không tính nguyên ví dụ này vào 8 test riêng. Mỗi người phụ trách 2 test; ba test có phép tính tay độc lập. So sánh hai thuật toán với nhau hữu ích nhưng chưa đủ nếu cả hai cùng mắc một lỗi.

```
python chay_kiem_thu.py
python dong_goi.py
```

Lệnh đầu lưu log thật, kể cả lỗi. Lệnh sau chỉ chạy khi nhóm đã có báo cáo, bài tay, slide, README và kết quả. Không ghi PASS khi chưa chạy.

Các tệp cần điền hoặc viết

Chọn mẫu Word/LaTeX trong `MauBaoCao/`, xuất `BaoCao.pdf`. Viết `TinhTay.pdf` và `ThuyetTrinh.pptx`. Điền hai mẫu `README_NOP_MAU.md` và `DongGop_MAU.md`; lưu thành `README_NOP.md` và `DongGop.md`. Hoàn thiện dữ liệu riêng và kiểm thử riêng.

Giữ đủ tệp Python phụ thuộc. Không nộp cache hoặc môi trường Anaconda. Sau đóng gói, giải nén và chạy README từ đầu trên máy thành viên khác. Nếu thêm thư mục mô-đun riêng, kiểm ZIP có đủ tệp cần import; công cụ có hỗ trợ thư mục `python/` do nhóm tự tạo.

Chuẩn bị bảo vệ

Chọn một nhánh cắt để giải thích bằng tay; một test bắt lỗi; một thay đổi utility hoặc thứ tự để dự đoán rồi chạy. Trong 15–20 phút, cả bốn người đều phải trả lời và chỉ mã, không chỉ một người bấm chạy.